Project Development Plan

This document provides an exhaustive development plan for the web application, structured around agile principles. We'll use **user stories** to define features from the end-user perspective, informing acceptance criteria, implementation steps, and testing. The project follows the **MVC (Model-View-Controller)** pattern:

- **Models**: Handle data logic and PostgreSQL interactions (SQLAlchemy).
- **Views**: React components for the frontend (graph canvas, forms, tables), Jinja templates for backend-rendered pages like /super_admin and /customization.
- **Controllers**: Flask routes/API endpoints (RESTful for React integration).

Git Workflow: Use Git for SCM. `main` branch protected. Work in feature branches (e.g., `git checkout -b feature/sprint-1-master-data`). PRs for review. Conventional commits (e.g., "feat: add user sync endpoint"). Repository structure includes README.md with setup instructions, .gitignore for node_modules/venv, and requirements.txt/requirements-dev.txt.

Code Documentation: All code will have docstrings, inline comments, and file headers. Example:

```python
# app/models/user.py
"""User model for authentication and roles."""

from app import db
from werkzeug.security import generate_password_hash, check_password_hash

class User(db.Model):
    """Represents a user in the system.

    Attributes:
        id (int): Primary key.
        username (str): Unique username.
        password_hash (str): Hashed password.
        role (str): 'user' or 'super_admin'.
    """
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True, nullable=False)
    password_hash = db.Column(db.String(128))
    role = db.Column(db.String(64), default='user')

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)
```

Testing: Integrated in every sprint. Use pytest for backend (unit/integration), Jest for frontend (unit). Small datasets (3–5 nodes/graphs). Coverage goal: 80%. Run with `pytest --cov=app`. CI/CD with GitHub Actions for auto-testing on PRs.

Deployment Matching: Develop on local Linux VMs (e.g., Ubuntu 22.04 LTS to match Digital Ocean droplets/App Platform). Use Docker-compose for local setup (Flask, React, Postgres). This mirrors production on Digital Ocean App Platform (deploys Flask as web service, React as static site, managed Postgres DB). From web search results, App Platform handles this seamlessly (build from GitHub, auto-detect Flask/React, attach Postgres). We'll include a `app.yaml` for App Platform in the repo.

Admin Endpoints: No separate app engine needed. All admin/customization endpoints are part of the same Flask app. Use blueprints for organization (e.g., `admin_bp = Blueprint('admin', __name__)`). Role checks in decorators (e.g., `@requires_role('super_admin')`).

Frontend: React 18 with Vite for build, React Flow 12.10.0 (latest as of Feb 2026 per web search, MIT license — no commercial restrictions). Fetch APIs for backend communication. No GUI testing in core suite — manual/smoke for canvas.

Sprints: Granular, 1–2 week cycles. Each sprint starts with 3–5 user stories, acceptance criteria, step-by-step implementation, code snippets (Python/JS), testing plan, and success metrics. Refactor as needed based on stories (e.g., if sync is slow, refactor in next sprint).

Sprint 0: Project Setup & Foundation

User Stories:

1. As a super_admin, I can set up the local development environment so I can run the app on my Linux VM.
2. As a developer, I can initialize the Git repo and push code so I can track changes using SCM.
3. As a super_admin, I can log in to the app so I can access protected endpoints like /super_admin.
4. As a developer, I can run basic tests to ensure the foundation is stable.

Acceptance Criteria:

- Local Linux VM matches Digital Ocean (Ubuntu 22.04, Python 3.11, Node 20, Postgres 16).
- App runs with `docker-compose up` (Flask on port 5000, React on 3000, Postgres on 5432).
- Basic login/logout works.
- Git repo has main branch, .gitignore, README with setup/deploy instructions.
- Tests pass with `pytest` and `npm test`.

Step-by-Step Implementation:

1. **Set up local Linux VM**:
    - Install Ubuntu 22.04 LTS (matches Digital Ocean).
    - Install prerequisites: `sudo apt update && sudo apt install python3-pip postgresql postgresql-contrib libpq-dev git docker-compose node20 npm`.
    - Create VM user matching your Digital Ocean droplet user.

2. **Initialize Git repo**:

   - `mkdir project && cd project`
   - `git init`
   - Create .gitignore:

```
text

# .gitignore
__pycache__/
*.pyc
*.pyo
*.egg-info/
.env
venv/
node_modules/
dist/
*.log
/data/*
```

   - Commit: `git add .gitignore && git commit -m "chore: initialize gitignore"`

3. **Flask backend skeleton**:

   - Create `requirements.txt`:

```
text

flask==3.0.3
flask-login==0.6.3
sqlalchemy==2.0.25
psycopg2-binary==2.9.9
werkzeug==3.0.1
```

- Create `app/__init__.py`:

```python
# app/__init__.py
"""Initialize the Flask app."""

from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager

db = SQLAlchemy()
login_manager = LoginManager()

def create_app():
    app = Flask(__name__)
    app.config.from_pyfile('config.py')  # Load from config.py

    db.init_app(app)
    login_manager.init_app(app)

    from app.models import models  # Import models
    from app.controllers.auth import auth_bp  # Import blueprints

    app.register_blueprint(auth_bp)

    return app
```

- Create `app/config.py`:

```python
# app/config.py
"""App configuration."""

import os

SECRET_KEY = os.environ.get('SECRET_KEY') or 'dev_key'
SQLALCHEMY_DATABASE_URI =
'postgresql://postgres:pass@localhost:5432/app_db'
SQLALCHEMY_TRACK_MODIFICATIONS = False
```

- Commit: `git add . && git commit -m "feat: flask skeleton"`

4. **Postgres setup**:

   - `docker-compose.yml` :

```
text

version: '3'
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: pass
    ports:
      - 5432:5432
  backend:
    build: .
    ports:
      - 5000:5000
    depends_on:
      - db
    environment:
      DATABASE_URL: postgresql://postgres:pass@db:5432/app_db
  frontend:
    build: frontend/
    ports:
      - 3000:3000
```

   - Run `docker-compose up` to test.

5. **React frontend skeleton**:

   - `cd frontend && npx create-vite@latest . --template react`
   - Add React Flow: `npm install @xyflow/react@12.10.0`
   - Basic App.js with React Flow canvas:

```jsx
// frontend/src/App.js
import React from 'react';
import { ReactFlow, Controls } from '@xyflow/react';
import '@xyflow/react/dist/style.css';

const App = () => (
  <ReactFlow defaultNodes={[]} defaultEdges={[]} fitView>
    <Controls />
  </ReactFlow>
);

export default App;
```

   - Commit: `git add frontend && git commit -m "feat: react frontend skeleton"`

6. **Basic auth controller**:

   - `app/controllers/auth.py` :

```
Python

# app/controllers/auth.py
"""Authentication routes."""

from flask import Blueprint, request, flash, redirect, url_for
from flask_login import login_user, logout_user, login_required
from app.models.user import User

auth_bp = Blueprint('auth', __name__)

@auth_bp.route('/login', methods=['GET', 'POST'])
def login():
    # Implementation...

@auth_bp.route('/logout')
@login_required
def logout():
    logout_user()
    return redirect(url_for('auth.login'))
```

   - Commit: `git commit -m "feat: basic auth"`

7. **Testing setup**:

- `pytest.ini`:

```
text

[pytest]
testpaths = tests
```

- `tests/conftest.py`:

```
Python

# tests/conftest.py
import pytest
from app import create_app, db

@pytest.fixture
def app():
    app = create_app()
    app.config.update({
        "TESTING": True,
        "SQLALCHEMY_DATABASE_URI":
"postgresql://postgres:pass@localhost:5432/test_db"
    })
    with app.app_context():
        db.create_all()
        yield app
        db.drop_all()
```

- Sample test:

```
Python

# tests/unit/test_models.py
def test_user_creation(app):
    with app.app_context():
        user = User(username='test', role='user')
        user.set_password('pass')
        db.session.add(user)
        db.session.commit()
        assert User.query.first().check_password('pass')
```

- Commit: `git commit -m "test: initial test suite"`

Success Metrics:

- App runs locally on Linux VM.
- Login works, tests pass 100%.
- Git repo ready for collaboration.

Sprint 1: Master Data & Super Admin Interface (User Stories & Steps)

User Stories:

1. As a super_admin, I can CRUD master device_types so I can update the library.
2. As a super_admin, I can CRUD master cable_lengths so I can manage dropdown options.
3. As a developer, I can test the master data endpoints to ensure they are secure.

Acceptance Criteria:

- Only super_admin can access /super_admin.
- Forms validate required fields.
- Tests cover 80% of models/controllers.

Step-by-Step Implementation:

1. Add models for master tables (e.g., `app/models/master.py`):

```python
# app/models/master.py
"""Master library models."""

from app import db

class DeviceType(db.Model):
    """Master device types."""
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(64), unique=True, nullable=False)
    color = db.Column(db.String(7), nullable=False)  # hex
    thumbnail = db.Column(db.String(128))  # filename
```

   - Similar for other master tables.

2. Super admin blueprint:

- `app/controllers/super_admin.py` :

```
Python

# app/controllers/super_admin.py
from flask import Blueprint, render_template, request, redirect,
url_for
from app.models.master import DeviceType
from app import db
from app.decorators import requires_super_admin

super_admin_bp = Blueprint('super_admin', __name__,
url_prefix='/super_admin')

@super_admin_bp.route('/device_types', methods=['GET', 'POST'])
@requires_super_admin
def device_types():
    if request.method == 'POST':
        # Add new
        name = request.form['name']
        color = request.form['color']
        thumbnail = request.files['thumbnail'].filename if
'thumbnail' in request.files else None
        new_type = DeviceType(name=name, color=color,
thumbnail=thumbnail)
        db.session.add(new_type)
        db.session.commit()
        return redirect(url_for('super_admin.device_types'))
    types = DeviceType.query.all()
    return render_template('super_admin/device_types.html',
types=types)
```

- Template `templates/super_admin/device_types.html` with form.

3. Decorator for role check:

```
Python

# app/decorators.py
from flask_login import current_user
from functools import wraps
from flask import abort

def requires_super_admin(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if not current_user.is_authenticated or current_user.role !=
'super_admin':
            abort(403)
        return f(*args, **kwargs)
    return decorated
```

4. Testing:

    - Add tests for CRUD, role denial (e.g., 403 on unauthorized access).

Success Metrics:

- Super_admin can manage master data.
- Tests pass, coverage >70%.

Sprint 2: User Onboarding, Sync, & Customization

User Stories:

1. As a user, I can sync master library so I have the latest nodes.
2. As an org admin, I can customize branding so exports look professional.
3. As a developer, I can test sync without conflicts.

Acceptance Criteria:

- Sync copies master to user rows without overwriting customs.
- /customization form saves branding + logo.
- Tests simulate sync scenarios.

Step-by-Step Implementation:

1. Add user-scoped tables (e.g., `user_device_types` with user_id, admin_source).
2. Sync route:
    - `app/controllers/sync.py` :

```
Python

# app/controllers/sync.py
from flask_login import current_user
from app.models.master import DeviceType
from app.models.user import UserDeviceType
from app import db

@sync_bp.route('/sync', methods=['POST'])
def sync():
    master_types = DeviceType.query.all()
    for mt in master_types:
        existing =
UserDeviceType.query.filter_by(user_id=current_user.id,
source_id=mt.id).first()
        if not existing or (existing and existing.admin_source):
            # Add or update
            user_type = UserDeviceType(user_id=current_user.id,
name=mt.name, color=mt.color, thumbnail=mt.thumbnail,
admin_source=True, source_id=mt.id)
            db.session.add(user_type)
    db.session.commit()
    current_user.last_synced_at = datetime.utcnow()
    db.session.commit()
    return {'status': 'success'}
```

3. Customization route as previously described.
4. Testing: Mock master data, assert user rows + flags.

Success Metrics:

- User syncs, edits (forks), branding form works.
- Tests pass.

Sprint 3: Graph Canvas & Node Addition

User Stories:

1. As a user, I can create a graph so I can diagram setups.
2. As a user, I can add nodes from library so I can build my system.
3. As a developer, I can test graph save/load.

Acceptance Criteria:

- React Flow canvas loads/saves graph_json.
- Add existing menu categorized.
- Node component shows notes, branding if org.

Step-by-Step Implementation:

1. API for graphs (POST /graphs, GET /graphs/:id).
2. React Flow setup:
    - `frontend/src/GraphCanvas.js`:

```jsx
// frontend/src/GraphCanvas.js
import React, { useState } from 'react';
import ReactFlow, { Controls, useNodesState, useEdgesState } from
'@xyflow/react';
import '@xyflow/react/dist/style.css';

const GraphCanvas = () => {
  const [nodes, onNodesChange] = useNodesState([]);
  const [edges, onEdgesChange] = useEdgesState([]);
  // Fetch/save logic here
  return (
    <ReactFlow
      nodes={nodes}
      edges={edges}
      onNodesChange={onNodesChange}
      onEdgesChange={onEdgesChange}
      fitView
    >
      <Controls />
    </ReactFlow>
  );
};

export default GraphCanvas;
```

3. Custom node component for rendering.
4. Testing: Pytest for API, Jest for component snapshot.

Success Metrics:

- Create/add nodes, save graph.
- Tests pass.

Sprint 4: Connections, Cable Lengths, & Tables

User Stories:

1. As a user, I can connect pins so I can wire diagrams.
2. As a user, I can set cable lengths so I can plan inventory.
3. As a user, I can view/export tables so I can document.

Acceptance Criteria:

- Double-click edge modal with lengths dropdown.
- Tables in 2-column format, CSV export.
- Inventory excludes system internals.

Step-by-Step Implementation:

1. Edge creation in React Flow (onConnect handler).
2. Double-click: Custom onEdgeClick to open modal, save to edge.data.
3. Table view in React (use TanStack Table for sorting).
4. Testing: Pytest for inventory logic, Jest for modal.

Success Metrics:

- Wire, set lengths, export.

Sprint 5: Systems & Org Enhancements

User Stories:

1. As a user, I can create systems so I can group nodes.
2. As an org admin, I can invite users so I can manage teams.

Acceptance Criteria:

- Nested React Flow in modal for systems.
- Org table + invite stub.

Step-by-Step Implementation:

1. Systems API.
2. Nested React Flow component.
3. Org management in /customization.
4. Testing: Integration for subgraph.

Success Metrics:

• Create system, add to graph.

Sprint 6: Customizations & Simplifications

User Stories:

1. As a user, I can use loop-through connections so I can model real setups.
2. As a user, I can assign switch ports so I can track network.

Acceptance Criteria:

• Loop edges with labels.

• Network handles positioned as per hack.

Step-by-Step Implementation:

1. Custom edge type for loops.
2. Custom handle positions in node component.
3. Testing: Manual visual + Jest snapshots.

Success Metrics:

• Hacks work.

This is exhaustive but actionable. For Digital Ocean: Use App Platform (from search: deploy Flask as service, React as static, Postgres attached). Local VMs: Ubuntu, Docker to match.

Do we need separate app engine? No — single Flask app handles all.

Ready for Sprint 0 code generation?